

陈启航

男 1994 年 8 月 23 日 现居深圳

电话: 13928514763 邮箱: 3690738212@qq.com

求职意向: Python 后端开发工程师 | 期望薪资: 10K-20K

个人总结

拥有 5 年 Python 后端开发经验, 基础扎实, 深入理解进程、线程及协程模型, 在实际项目中多次运用异步编程解决高并发场景下的性能瓶颈。熟练使用 Django 和 FastAPI 框架, 严格遵循 RESTful 规范进行 API 设计与开发。熟悉 MySQL 索引优化与慢查询排查, 擅长结合 Redis 设计多级缓存策略以降低数据库压力。具备 RabbitMQ 和 Kafka 的实际项目使用经验。编码习惯良好, 在团队中推动过代码评审与单元测试规范落地, 了解常用设计模式并能在业务场景中灵活运用。逻辑分析能力强, 始终坚持用数据驱动决策, 通过全链路复盘定位问题并推动系统持续优化。

工作经历

深圳某智能科技公司 | Python 后端开发工程师 | 2022 年 4 月 - 至今

负责公司数据中台核心服务的后端开发, 参与高并发分布式系统的设计与迭代, 与产品、算法及运维团队紧密协作, 推动业务需求快速落地。

基于 FastAPI 框架搭建微服务架构, 严格遵循 RESTful API 设计规范, 制定统一的请求响应格式、错误码体系及接口版本管理策略, 日均处理请求量超过 300 万次。

深入应用 Python 异步编程模型 (async/await), 利用协程特性将核心数据查询接口的并发处理能力提升 3 倍以上。

主导数据清洗管道从同步到异步的重构, 引入 Celery+RabbitMQ 作为分布式任务调度层, 解耦数据处理链路, 批量处理时长从 47 分钟压缩至 8 分钟以内。

熟练使用 MySQL 进行复杂业务查询开发, 针对下游报表服务频繁超时问题, 通过慢查询日志分析和执行计划调优, 联合 DBA 优化索引策略, 查询性能提升近 20 倍。

设计 Redis 多级缓存方案, 将高频访问数据分层缓存, 数据库读压力降低约 55%, 保障了高并发场景下的系统稳定性。

运用工厂模式与策略模式重构了多数据源接入模块, 使新增一种数据源的开发周期从 3 天缩短至半天, 代码可维护性显著提升。

上海某信息技术有限公司 | 后端开发工程师 | 2019 年 7 月 - 2022 年 2 月

参与电商 SaaS 平台的后端开发, 使用 Django 框架进行业务功能模块的设计与实现。

独立负责订单管理、库存同步等微服务的 RESTful API 设计与开发, 编写清晰的接口文档, 与前端和产品团队高效协作。

为订单查询接口设计 Redis 缓存策略, 在促销活动高并发期间数据库压力降低约 40%, 接口 P99 响应时间从 2.1 秒降至 320 毫秒。

使用 RabbitMQ 实现订单状态变更的异步通知机制, 确保库存扣减、物流推送等下游操作最终一致性, 消除同步调用造成的超时雪崩风险。

推动团队引入代码评审机制和单元测试规范, 使用单例模式统一管理配置资源、工厂模式解耦对象创建逻辑, 项目缺陷率在半年内下降约 35%。

通过 Kafka 消费业务埋点日志, 搭建用户行为数据实时采集管道, 为运营团队提供分钟级的数据看板支持。

项目亮点

在数据中台项目中, 针对下游报表服务频繁超时的问题, 通过全链路数据漏斗分析定位到慢 SQL 和索引缺失是关键瓶颈, 优化后查询性能提升近 20 倍。在电商 SaaS 平台中, 通过用户反馈和数据复盘定位到三个订单状态同步的关键体验断点, 设计 Webhook 回调加失败重试

方案，将物流信息更新延迟率从 8%降至 0.5%以下。

教育经历

华中科技大学 | 软件工程专业 | 本科 | 2015 年 9 月 - 2019 年 6 月

主修课程：数据结构与算法、操作系统、计算机网络、数据库原理、软件工程。在校期间参加全国大学生数学建模竞赛获省级二等奖，锻炼了逻辑分析与问题抽象能力。

专业技能

编程语言：精通 Python，深入理解进程、线程、协程模型及异步编程范式；了解 Golang 基础语法

Web 框架：熟练使用 Django 和 FastAPI，严格遵循 RESTful API 设计规范，有丰富的接口设计与开发经验

数据库与缓存：熟悉 MySQL 索引优化、慢查询排查与 SQL 调优；熟悉 Redis 缓存策略、分布式锁及应用场景

消息队列：有 RabbitMQ 和 Kafka 的实际项目使用经验，理解消息队列在解耦、削峰、异步处理中的典型应用场景

工程实践：编码习惯良好，了解工厂模式、单例模式、策略模式等常用设计模式；有推动代码评审和单元测试落地的实践经验

逻辑分析与优化：擅长通过数据分析和全链路复盘定位性能瓶颈，用数据驱动系统持续优化